Name: Muhammad Talha Javed
Assignment: Pipelined Risc-V processor
Submitted to: Sir Muhammad Yousaf
Subject: Computer Architecture

Design and Implementation of a Five-Stage Pipelined RISC-V Processor Using Verilog

---

1. Introduction

This project implements a **5-stage pipelined RISC-V processor** in Verilog using **Xilinx Vivado**. The provided single-cycle processor was redesigned by dividing instruction execution into five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Pipeline registers were inserted between stages to allow multiple instructions to execute simultaneously. A forwarding unit and a hazard detection unit were implemented to handle data hazards and load-use hazards. The complete processor was verified through simulation using a custom testbench.

The completed design was verified through simulation using a custom testbench that executes a small RISC-V program containing arithmetic, logical, memory, branch, and jump instructions. The simulation results confirm the correct operation of all pipeline stages and hazard handling mechanisms.

2. Objectives

The main objectives of this project are:

- Convert the provided single-cycle RISC-V processor into a five-stage pipelined processor.
- Divide the processor into IF, ID, EX, MEM, and WB stages.
- Implement IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers.
- Pipeline both data and control signals between stages.
- Implement a forwarding unit to resolve Read-After-Write (RAW) hazards.
- Implement a hazard detection unit to resolve load-use hazards.
- Integrate all modules into a top-level processor.
- Verify the processor using simulation in Vivado.

These objectives directly correspond to the project tasks specified in the assignment.

3. Project Structure

To simplify the design and improve readability, the processor was implemented using a **modular architecture**. Each functional block of the processor was written as an independent Verilog module. Instead of placing all logic inside a single file, separate modules were created for each pipeline stage, pipeline register, hazard handling unit, memory, and arithmetic unit.

The top-level module (riscv_top) connects all these modules together to form the complete processor. This modular approach makes debugging easier and allows each stage to be verified independently before integrating the complete design.

The major modules implemented in this project are listed below.

Module	Function
fetch_stage	Instruction Fetch stage
prg_mem	Instruction Memory
IF_ID_reg	IF/ID Pipeline Register
reg_file	Register File
imm_gen	Immediate Generator
controller	Main Control Unit
alu_decoder	ALU Control
ID_EX_reg	ID/EX Pipeline Register
forwarding_unit	Data Hazard Resolution
alu	Arithmetic Logic Unit
branch_comp	Branch Comparator
EX_MEM_reg	EX/MEM Pipeline Register
data_mem	Data Memory
MEM_WB_reg	MEM/WB Pipeline Register
hazard_unit	Load-use Hazard Detection
riscv_top	Top-Level Processor
tb_riscv_top	Testbench

4. Overall Processor Architecture

The processor follows the standard **five-stage pipeline architecture**. During execution, multiple instructions are processed simultaneously, with each instruction occupying a different stage of the pipeline. This allows the processor to increase instruction throughput without significantly increasing hardware complexity.

The five pipeline stages are:

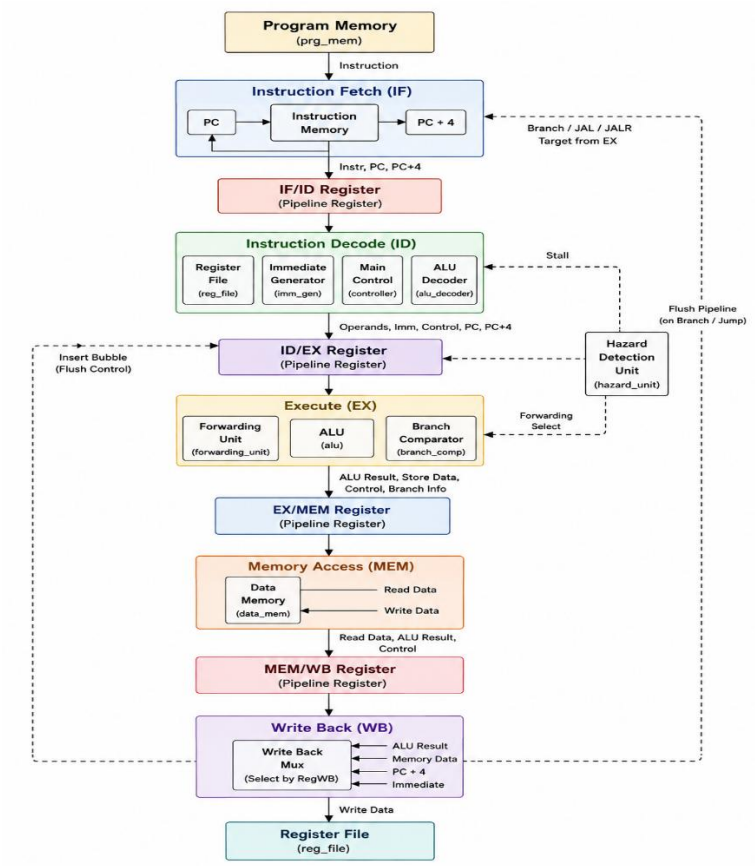
1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

Between every stage, a pipeline register stores intermediate data and control signals. These registers isolate the stages and ensure that each stage processes information from the previous clock cycle.

The processor also contains two additional units that improve correct execution:

- **Forwarding Unit** for resolving data hazards.
- **Hazard Detection Unit** for resolving load-use hazards through pipeline stalls.

The following diagram illustrates the overall flow of instruction execution through the processor.



The riscv_top module is responsible for instantiating each of these modules and connecting them together to form the complete pipelined processor.

5. Instruction Fetch (IF) Stage

The Instruction Fetch (IF) stage is responsible for fetching instructions from the program memory. It contains the **Program Counter (PC)**, which stores the address of the current instruction being executed. During normal operation, the PC increments by four bytes because each RISC-V instruction occupies one 32-bit word. If a branch or jump instruction is taken, the PC is updated with the corresponding target address instead of the next sequential address.

The fetch stage also receives the **stall** signal from the hazard detection unit. When a load-use hazard is detected, the PC is held constant, preventing the processor from fetching a new instruction until the hazard has been resolved.

Important Code Snippet

```
always @(posedge clk or negedge rst_n) begin
   if (!rst_n)
      PC <= 32'h0;
   else if (stall)
      PC <= PC;
   else if (jalr_sel)
      PC <= jalr_target;
   else if (branch_taken)
      PC <= branch_target;
   else
      PC <= PC + 4;
end
```

Explanation

This sequential block updates the Program Counter on every positive clock edge. During reset, the PC is initialized to address zero. When the hazard detection unit asserts the stall signal, the current PC value is retained, effectively pausing instruction fetch. If a JALR or branch instruction changes the program flow, the PC is loaded with the computed target address. Otherwise, the PC increments by four to fetch the next instruction.

The fetched instruction is read from the **Program Memory (prg_mem)**, which stores the hardcoded RISC-V test program used during simulation.

6. IF/ID Pipeline Register

The IF/ID pipeline register separates the Instruction Fetch and Instruction Decode stages. Its purpose is to store the fetched instruction along with the corresponding PC and PC+4 values so that the Decode stage can process them during the next clock cycle.

The register also supports **stall** and **flush** operations. During a stall, the stored values are preserved to prevent the Decode stage from receiving a new instruction. During a flush, all outputs are cleared, effectively inserting a **NOP (bubble)** into the pipeline. This mechanism is used to remove incorrectly fetched instructions after a branch or jump.

Important Code Snippet

```
always @(posedge clk or negedge rst_n) begin
   if (!rst_n) begin
      pc_out <= 32'd0;
      pc_plus4_out <= 32'd0;
      instr_out <= 32'd0;
   end
   else if (flush) begin
      pc_out <= 32'd0;
      pc_plus4_out <= 32'd0;
      instr_out <= 32'd0;
   end
```

Explanation

The pipeline register stores the outputs of the Fetch stage on every clock cycle. When a reset or flush occurs, the register outputs are cleared, which inserts a bubble into the pipeline. This prevents invalid instructions from progressing further through the processor.

7. Instruction Decode (ID) Stage

The Instruction Decode stage interprets the fetched instruction and prepares all information required for execution. This stage consists of four main modules:

- Register File
- Immediate Generator
- Controller
- ALU Decoder

The register file reads the source operands specified by the instruction. Simultaneously, the Immediate Generator extracts and sign-extends immediate values depending on the instruction

format. The Controller decodes the instruction opcode and generates all control signals required by the later pipeline stages, while the ALU Decoder determines the exact ALU operation based on the instruction type.

Together, these modules convert the binary instruction into operands, control signals, and immediate values that are passed to the Execute stage through the ID/EX pipeline register.

7.1 Register File

The Register File stores the processor's 32 general-purpose registers. It provides two asynchronous read ports and one synchronous write port. Register **x0** is permanently hardwired to zero, ensuring compliance with the RISC-V specification.

During the Decode stage, the register addresses (rs1 and rs2) are extracted from the instruction, and the corresponding register values are read immediately. During the Write Back stage, the destination register is updated with the computed result if the RegWrite signal is asserted.

Important Code Snippet

```
assign data1 = (addr1 == 5'd0) ? 32'h00000000 : mem[addr1];
assign data2 = (addr2 == 5'd0) ? 32'h00000000 : mem[addr2];
```

Explanation

The above logic ensures that register **x0** always returns zero, while all other registers return their stored values. This matches the RV32I architecture, where register zero cannot be modified.

7.2 Immediate Generator

Many RISC-V instructions require an immediate value instead of a second register operand. The Immediate Generator (imm_gen) extracts the immediate field from the instruction and sign-extends it to 32 bits. Since RISC-V uses different instruction formats (I-type, S-type, B-type, U-type, and J-type), the immediate value is generated differently depending on the instruction opcode.

The generated immediate is later used by the ALU for arithmetic operations, memory address calculations, branch target calculation, and jump instructions.

Important Code Snippet

```
case (opcode)
   7'b0010011, 7'b0000011, 7'b1100111:
      imm = {{20{instruction[31]}}, instruction[31:20]};

   7'b0100011:
      imm = {{20{instruction[31]}},
           instruction[31:25],
           instruction[11:7]};
```

Explanation

The Immediate Generator first identifies the instruction format using the opcode. It then extracts the corresponding bits from the instruction and performs sign extension to produce a 32-bit immediate value. This allows different instruction formats to be processed correctly without additional hardware.

7.3 Controller

The Controller is responsible for generating all control signals required by the processor. It examines the opcode of the current instruction and determines how the remaining stages of the pipeline should behave.

For example, arithmetic instructions enable register writing, load instructions enable memory reading, store instructions enable memory writing, and branch instructions activate branch control logic.

The generated control signals are passed through the pipeline registers so that every stage receives the correct control information.

Important Code Snippet

```
7'b0000011: begin      // Load
   SrcSel   = 1'b1;
   RegWrite = 1'b1;
   MemRead  = 1'b1;
   RegWB    = 2'b01;
end
```

Explanation

The above code shows the control signals generated for a **Load (LW)** instruction. Since the ALU must calculate the memory address using an immediate value, SrcSel is enabled. Memory reading and register writing are also enabled, while the write-back multiplexer is configured to select the data read from memory.

7.4 ALU Decoder

The ALU Decoder generates the control signal (AluSel) that determines which operation the ALU should perform. Unlike the main controller, which only identifies the instruction type, the ALU Decoder uses the opcode together with the funct3 and funct7 fields to distinguish between individual arithmetic and logical operations.

This allows instructions such as ADD, SUB, AND, OR, XOR, SLL, and SRL to use the same datapath while selecting different ALU operations.

Important Code Snippet

```
3'b000:
   AluSel = funct7_5 ? 4'b0001 : 4'b0000;
```

Explanation

When funct3 equals 000, the instruction may represent either **ADD** or **SUB**. The ALU Decoder checks bit 30 (funct7_5) to distinguish between the two instructions. If the bit is 0, the ALU performs addition; otherwise, it performs subtraction.

8. ID/EX Pipeline Register

The ID/EX pipeline register separates the Decode and Execute stages. After the instruction has been decoded, all operands, immediate values, destination register information, and control signals are stored in this register before being forwarded to the Execute stage.

In addition to normal data transfer, this register also supports **stall** and **flush** operations. During a load-use hazard, the hazard detection unit inserts a bubble by clearing the control signals while preserving the pipeline timing. Similarly, during a branch or jump, the register is flushed to remove incorrectly decoded instructions.

Important Code Snippet

```
if (!rst_n || flush) begin
   RegWrite_out  <= 1'b0;
   MemWrite_out  <= 1'b0;
   BranchSel_out <= 1'b0;
   MemRead_out   <= 1'b0;
end
```

Explanation

During a reset or pipeline flush, the control signals are cleared. This converts the instruction into a **NOP (No Operation)**, preventing it from modifying registers or memory while maintaining correct pipeline operation.

9. Execute (EX) Stage

The Execute stage performs all arithmetic, logical, and branch operations of the processor. It receives operands, immediate values, and control signals from the ID/EX pipeline register. Depending on the instruction type, the ALU performs the required operation, calculates memory addresses for load/store instructions, or computes the target address for branch and jump instructions.

Since this stage is responsible for instruction execution, it also contains the **Forwarding Unit** and **Branch Comparator**, which help resolve hazards and determine whether a branch should be taken.

9.1 Forwarding Unit

In a pipelined processor, data hazards occur when an instruction requires the result of a previous instruction before it has been written back to the register file. Instead of stalling the pipeline for every dependency, a forwarding unit is used to forward the latest available result directly to the ALU inputs.

The forwarding unit compares the source registers of the current instruction with the destination registers of instructions present in the EX/MEM and MEM/WB pipeline stages. If a match is found and register writing is enabled, the forwarded value is selected instead of the register file output.

Important Code Snippet

```
if (EX_MEM_RegWrite &&
   (EX_MEM_rd != 5'd0) &&
   (EX_MEM_rd == ID_EX_rs1))
   ForwardA = 2'b10;
```

Explanation

The above logic checks whether the destination register stored in the EX/MEM pipeline register matches the first source register of the current instruction. If the previous instruction writes to the same register, the ALU input is taken directly from the EX/MEM stage instead of waiting for the value to be written back to the register file.

This forwarding mechanism removes most Read-After-Write (RAW) hazards and significantly improves pipeline performance.

9.2 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit performs all arithmetic and logical operations required by the processor. The operation to be performed is determined by the AluSel control signal generated by the ALU Decoder.

The ALU supports arithmetic operations such as addition and subtraction, logical operations including AND, OR, and XOR, shift operations, and comparison operations required by branch instructions.

Important Code Snippet

```
case (AluSel)
   4'b0000: data_out = data1 + data2;
   4'b0001: data_out = data1 - data2;
   4'b1000: data_out = data1 | data2;
   4'b1001: data_out = data1 & data2;
endcase
```

Explanation

The ALU selects the required operation based on the AluSel signal. For example, an ADD instruction performs addition, while an AND instruction performs a bitwise AND operation. The output of the ALU is forwarded to the Memory stage or written back to the register file depending on the instruction type.

9.3 Branch Comparator

The Branch Comparator determines whether a branch instruction should be taken. It compares the values of the source registers according to the branch type specified by the instruction (BEQ, BNE, BLT, BGE, etc.).

The comparison result is sent to the control logic, which decides whether the Program Counter should continue sequential execution or jump to the branch target.

Important Code Snippet

```
case (br_type)
   3'b000: br_taken = (data1 == data2);
   3'b001: br_taken = (data1 != data2);
```

Explanation

The comparator checks the branch type and performs the corresponding comparison. For a BEQ instruction, the branch is taken only if both operands are equal. Similar comparisons are performed for the remaining branch instructions supported by the processor.

10. EX/MEM Pipeline Register

The EX/MEM pipeline register stores the outputs generated during the Execute stage before they are transferred to the Memory stage. These outputs include the ALU result, forwarded store data, destination register number, and all control signals required during memory access and write-back.

This register ensures that the Memory stage processes stable data generated during the previous clock cycle while allowing the Execute stage to begin processing the next instruction.

Important Code Snippet

```
alu_result_out <= alu_result_in;
rs2_data_out   <= rs2_data_in;
rd_out         <= rd_in;

RegWrite_out   <= RegWrite_in;
MemWrite_out   <= MemWrite_in;
MemRead_out    <= MemRead_in;
```

Explanation

At every rising clock edge, the EX/MEM pipeline register transfers the execution results and associated control signals to the Memory stage. These stored values are then used for memory read/write operations and later passed to the Write Back stage.

11. Memory Access (MEM) Stage

The Memory Access stage is responsible for performing data memory operations. For **load (LW)** instructions, the stage reads data from memory using the address calculated by the ALU. For **store (SW)** instructions, it writes the value of the source register into the specified memory location. Arithmetic and logical instructions simply pass the ALU result through this stage without accessing memory.

The data memory is implemented as a 1024-word memory array with synchronous write and asynchronous read operations.

Important Code Snippet

```
always @(posedge clk or negedge rst_n) begin
   if(wr_en)
      mem[addr[31:2]] <= data_in;
end
assign data_out = mem[addr[31:2]];
```

Explanation

The write operation occurs only on the positive edge of the clock when the write enable (`wr_en`) signal is asserted. Since each memory location stores a 32-bit word, the address is divided by four (`addr[31:2]`) to convert the byte address into a word index. The read operation is asynchronous, allowing data to be available immediately after the address changes.

12. MEM/WB Pipeline Register

The MEM/WB pipeline register stores the outputs of the Memory stage before they are sent to the Write Back stage. Depending on the instruction, these outputs may include the memory read data, ALU result, immediate value, or PC+4 value.

The register also stores the destination register number and the write-back control signals required for updating the register file.

Important Code Snippet

```
read_data_out  <= read_data_in;
alu_result_out <= alu_result_in;
rd_out         <= rd_in;
RegWrite_out   <= RegWrite_in;
RegWB_out      <= RegWB_in;
```

Explanation

This pipeline register transfers the outputs of the Memory stage to the Write Back stage on every clock cycle. It preserves both data and control signals so that the correct result is written back to the destination register.

13. Write Back (WB) Stage

The Write Back stage is the final stage of the pipeline. It selects the correct result and writes it into the destination register of the Register File. Depending on the instruction type, the value written back may be the ALU result, memory data, PC+4, or an immediate value.

The selection is controlled by the RegWB signal generated by the control unit.

Important Code Snippet

```
assign wb_data =
   (RegWB_MEM_WB == 2'b01) ? read_data_MEM_WB :
   (RegWB_MEM_WB == 2'b10) ? pc_plus4_MEM_WB :
   (RegWB_MEM_WB == 2'b11) ? imm_MEM_WB :
                     alu_result_MEM_WB;
```

Explanation

A multiplexer selects the appropriate value based on the RegWB control signal. For load instructions, memory data is written back. Arithmetic instructions write the ALU result, while JAL/JALR instructions write the return address (PC+4). LUI instructions directly write the immediate value.

14. Hazard Detection Unit

Although the forwarding unit resolves most data hazards, it cannot resolve **load-use hazards** because the required data is not available until the Memory stage. The Hazard Detection Unit detects this situation and temporarily stalls the pipeline.

When a load-use hazard occurs, the Program Counter and IF/ID pipeline register are frozen, while the ID/EX register inserts a bubble (NOP). After one clock cycle, normal execution resumes.

Important Code Snippet

```
if (ID_EX_MemRead &&
   (ID_EX_rd != 5'd0) &&
   ((ID_EX_rd == IF_ID_rs1) ||
    (ID_EX_rd == IF_ID_rs2)))
begin
   stall = 1'b1;
end
```

Explanation

The hazard unit checks whether the instruction currently in the Execute stage is a load instruction and whether its destination register matches either source register of the instruction currently in the Decode stage. If both conditions are satisfied, the stall signal is asserted to prevent incorrect execution.

15. Top-Level Integration

The entire processor is integrated in the riscv_top module. This module instantiates all pipeline stages, memories, pipeline registers, forwarding logic, and hazard detection logic, connecting them together to form the complete five-stage pipelined processor.

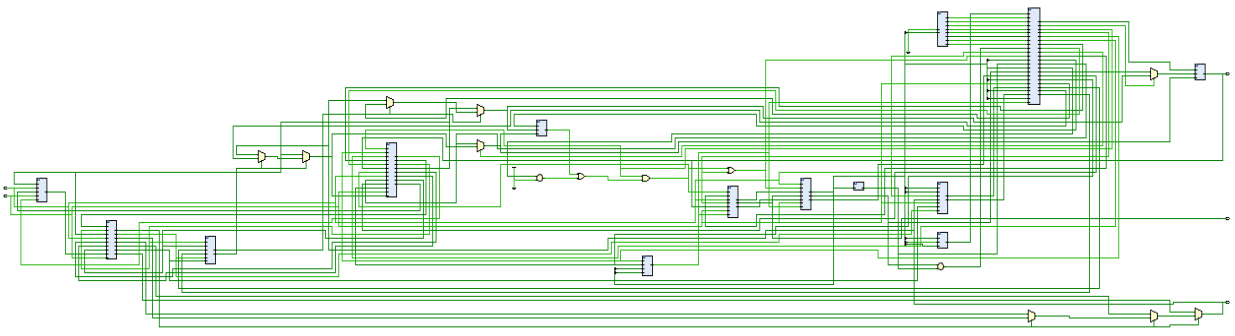
In addition to connecting the datapath, the top-level module also generates the pipeline flush signal for branch and jump instructions and routes the probe signals used by the testbench for simulation.

Project Summary | Schematic | tb_riscv_top.v | **riscv_top.v** | fetch_stage.v | prg_mem.v

C:/Users/hp/Documents/project_3___/project_3___.srcs/sources_1/imports/Design Sources/riscv_top.v

```
1  `timescale 1ns / 1ps
2  module riscv_top (
3      input  wire        clk,
4      input  wire        rst_n,
5      output wire [31:0] probe_pc,
6      output wire [31:0] probe_alu_out,
7      output wire [31:0] probe_wb_data
8  );
9      wire [31:0] pc_IF, pc_plus4_IF, instr_IF;
10     wire [31:0] pc_ID, pc_plus4_ID, instr_ID;
11
12
13     wire [31:0] pc_ID_EX, pc_plus4_ID_EX, rs1_data_ID_EX, rs2_data_ID_EX, imm_ID
14                 branch_target_ID_EX, instr_ID_EX;
15     wire [4:0]  rs1_ID_EX, rs2_ID_EX, rd_ID_EX;
16     wire [2:0]  funct3_ID_EX;
17     wire [3:0]  AluSel_ID_EX;
18     wire        SrcSel_ID_EX, MemWrite_ID_EX, RegWrite_ID_EX, AuipcSel_ID_EX,
19                 BranchSel_ID_EX, JalrSel_ID_EX, MemRead_ID_EX;
20     wire [1:0]  RegWB_ID_EX;
21     wire [31:0] alu_result_EX_MEM, rs2_data_EX_MEM, pc_plus4_EX_MEM, imm_EX_MEM;
22     wire [4:0]  rd_EX_MEM;
```

28 Cells 98 I/O Ports 1202 Nets



16. Testbench

To verify the functionality of the processor, a dedicated Verilog testbench (tb_riscv_top) was developed. The testbench generates the system clock and reset signals, instantiates the top-level processor, and monitors important internal signals during simulation.

Initially, the reset signal is asserted to initialize the processor. After a few clock cycles, the reset is released, allowing the processor to begin executing the instructions stored in the program memory. The testbench waits until the Program Counter reaches the final instruction of the test program before terminating the simulation.

In addition, a watchdog timer is included to prevent the simulation from running indefinitely if an error occurs, such as an incorrect branch target or pipeline control issue.

Important Code Snippet

```
initial begin
   clk = 0;
   forever #5 clk = ~clk;
end

initial begin
   rst_n = 0;
   #25;
   rst_n = 1;
end
```

Explanation

The first block generates a clock with a period of **10 ns**, while the second block applies an active-low reset to initialize the processor before instruction execution begins.

Important Code Snippet

```
wait(probe_pc == 32'h00000064);
#50;
$finish;
```

Explanation

The simulation waits until the processor reaches the last instruction of the test program (located at address 0x64). After allowing a few additional clock cycles for the pipeline to complete execution, the simulation terminates successfully.

17. Simulation Results

The processor was simulated using **Xilinx Vivado Simulator** to verify the functionality of all pipeline stages. The simulation confirmed that instructions moved correctly through the IF, ID, EX, MEM, and WB stages while the forwarding unit and hazard detection unit operated as expected.

The hardcoded program stored in the instruction memory includes arithmetic operations, logical operations, memory access instructions, conditional branches, jump instructions, and immediate instructions. Successful execution of this program verified that the processor correctly handled both normal instruction flow and pipeline hazards.

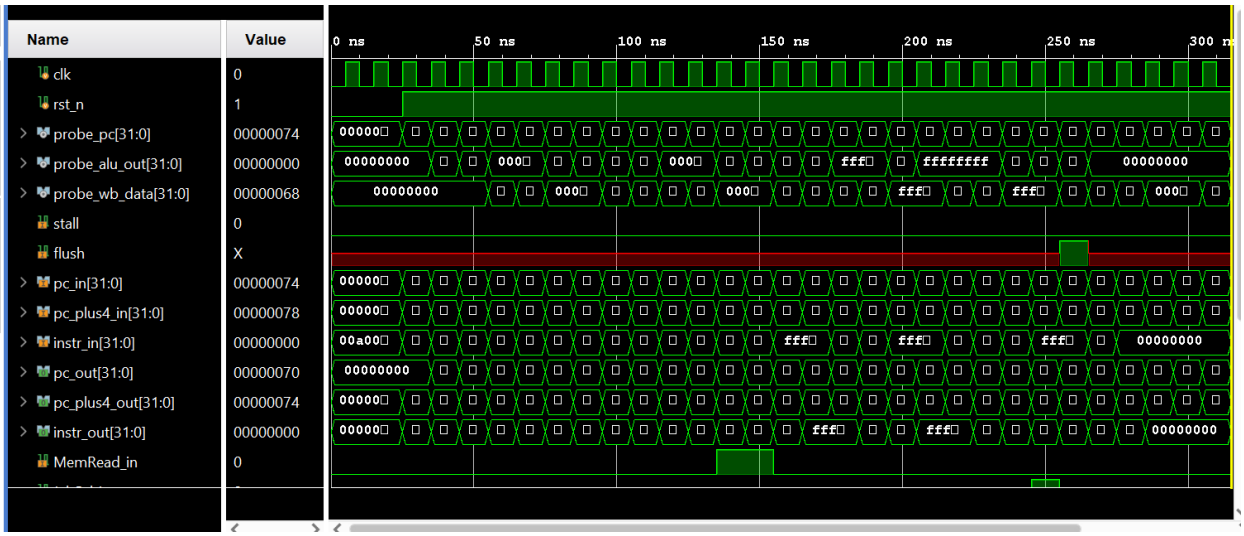
During simulation, the following signals were monitored:

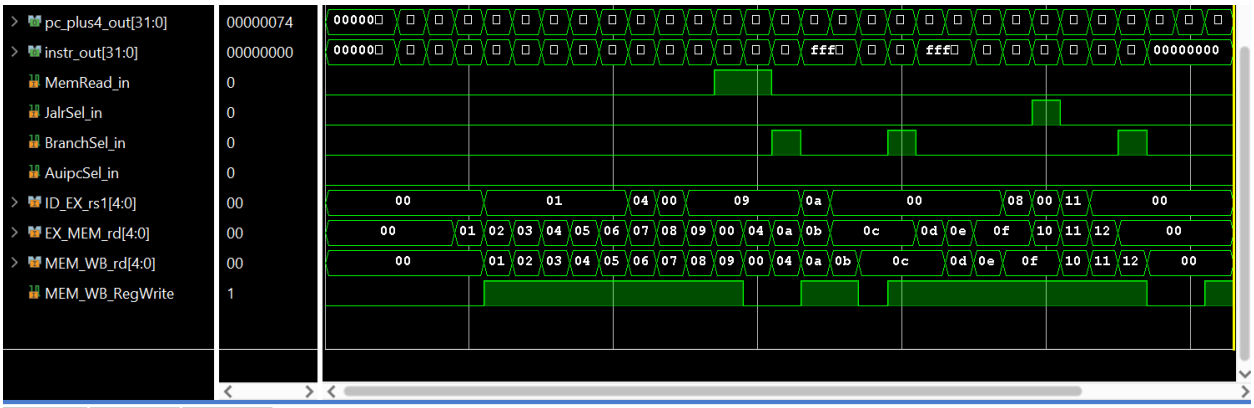
- Program Counter (PC)
- Instruction
- ALU Output
- Write Back Data
- Stall Signal
- Branch Taken Signal
- ForwardA
- ForwardB

The waveform showed that:

- The Program Counter incremented correctly during sequential execution.
- Forwarding eliminated unnecessary stalls for dependent ALU instructions.
- A single-cycle stall was inserted for load-use hazards.
- Branch and jump instructions correctly flushed invalid pipeline instructions.
- Register values were updated correctly during the Write Back stage.

Behavioral Simulation





18. Conclusion

A five-stage pipelined RISC-V processor was successfully implemented in Verilog using Xilinx Vivado. The original single-cycle processor was redesigned by dividing instruction execution into the Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back stages, with pipeline registers inserted between each stage.

To ensure correct execution, a forwarding unit was implemented to resolve most Read-After-Write (RAW) hazards, while a hazard detection unit introduced pipeline stalls for load-use hazards. Control hazards caused by branch and jump instructions were handled by flushing incorrect instructions from the pipeline.

Simulation results verified the correct operation of the processor, including arithmetic operations, memory access, branching, and hazard handling. The modular implementation also makes the design easier to understand, debug, and extend for future enhancements such as branch prediction, cache memory, or additional RISC-V instructions.